Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.9 / COMPANION WORKBOOK

Stand up the business services

Files, database, licensing & web — by documented paths

Give the users what the network exists for: SBD-FS01's role-based shares over verified SMB security, SBD-WEB01 serving internal HTTPS from a hardened, domain-joined Linux Apache, SBD-APPS01's licensing service and application API, and SBD-SQL01 running as a managed identity behind exactly one caller — every hop a documented, proven register row.

MATERIALS EDITION 1.3 / COURSE 1.9

Updated 9 October 2026 · sbd-business-services

NAME _____ / DATE _____

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Design the service paths and identities

02 Build SBD-FS01 with least-privilege shares

03 Build SBD-SQL01 behind its applications

04 Build SBD-APPS01 and SBD-WEB01

05 Reconcile, scan, and hand off

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

The users finally get what the network exists for: files, licensing, and an internal web application with its application tier and database. The model's rule governs the whole course — permit only documented business service paths — so every service arrives with its register rows, its service identity, and its certificate, and nothing reaches it any other way.

Prerequisites

- ☐ Complete courses 1.4–1.8: working PKI services and the watched, managed network with the ring/source rules in force, on the routed, segmented VLANs.
- ☐ Provision the VLAN 40 hosts: SBD-FS01 (10.0.40.10), SBD-SQL01 (10.0.40.20), SBD-APPS01 (10.0.40.30), and SBD-WEB01 (10.0.40.40, Linux with Apache, joined to the sbd.test domain with SSSD like SBD-SPLUNK01 and SBD-NESSUS01).
- ☐ Have the network's service-path register and the course 1.4 SBD Server Authentication template ready — every service here gets a documented path and a real certificate.

What this course builds

- Serve files through role-group permissions with verified SMB signing and encryption.
- Run the database as a managed identity, certificate-encrypted, with SBD-APPS01 as its only caller.
- Prove the fleet → web → application → database chain with every hop registered and every shortcut denied.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-FS01 — name, address, OS build	
SBD-SQL01 — name, address, OS build	
SBD-APPS01 — name, address, OS build	
SBD-WEB01 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 30 MIN

Design the service paths and identities

> **WHY THIS MATTERS** Four services, and before any of them exist: who talks to what over which port, which identity each service runs as, and which groups own access — the register and the directory answer all three before a role installs.

In this lesson you will

- Write the register rows for every business service path.
- Choose managed service identities instead of password accounts.
- Define the access groups that will own each service's data.

Follow the build

1. Write the service-path rows: fleet to FS01 (SMB 445); fleet to APPS01 on its licensing port, recorded per application; fleet to WEB01 (HTTPS 443); WEB01 to APPS01 for the application API (HTTPS 443); and APPS01 — and only APPS01 — to SQL01 (TCP 1433 with TLS). Add an explicit deny from WEB01 to SQL01: the web tier has no database dependency. All four servers share VLAN 40, so the router never sees their traffic to each other — each server's host firewall enforces those rows. Everything else toward VLAN 40 stays denied, including endpoint-to-SQL directly.
2. Register WEB01's Linux maintenance rows as well: packages from SBD-REPO01 only (HTTP with GPG-signed packages) and syslog over TLS to the collector, SBD-SYSLOG01 (TCP 6514) — the same paths the other domain-joined Linux servers, SBD-SPLUNK01 and SBD-NESSUS01, use. Course 1.8's repository row named only those two because WEB01 did not exist yet.
3. Choose the service identities: group managed service accounts (gMSAs) for the SQL and application services, so passwords rotate themselves — and note delegated managed service accounts, new in current Windows Server releases, as the migration path for any legacy service account this lab later inherits.
4. Create the access groups under the course 1.3 Groups OU — per-share and per-application role groups — because permissions will be granted to roles, never to named humans.
5. Request the server certificates through the course 1.4 process: one SBD Server Authentication certificate each for SQL01, APPS01 (for its application API), and WEB01, enrolled from the template, with the publication site serving their revocation story.

Check your result

```
Get-ADServiceAccount -Filter * | Select-Object Name, Enabled
Get-ADGroup -SearchBase 'OU=Groups,OU=SBD,DC=sbd,DC=test' -Filter * | Select-Object Name
Test-NetConnection 10.0.40.20 -Port 1433   # expected before build: TcpTestSucceeded False
```

> **WHAT SUCCESS LOOKS LIKE** The register carries every service row — the SQL path restricted to APPS01 alone, the WEB01-to-SQL01 deny, and WEB01's repository and syslog rows — the gMSAs and role groups exist, and the three server certificates are issued and recorded.

Your record

Evidence to keep: Keep the service-path rows, the identity and group design, and the certificate enrollment records.

© 2026 Secure By Design Team LLC.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Who may reach SQL01 on 1433?

REMEMBER Every path is a row; the database hears only SBD-APPS01. Service identities are managed accounts, not passwords. Access belongs to roles; certificates are already in hand.

Microsoft · Group managed service accounts

Microsoft · What's new in Windows Server 2025

NIST · SP 800-53 Rev. 5 security controls

LESSON 02 / ABOUT 30 MIN

Build SBD-FS01 with least-privilege shares

> **WHY THIS MATTERS** The file server is where access control meets daily life. Build the shares against the role groups, require SMB signing on the server instead of trusting client defaults, and make 'who can read this' a one-lookup question.

In this lesson you will

- Create the shares with role-group permissions and no individual grants.
- Confirm the SMB signing and encryption posture end to end.
- Verify access and denial per role from the fleet.

Follow the build

1. Build SBD-FS01 in the Servers OU and create the planned shares, granting NTFS permissions to the lesson-1 role groups only — share permissions stay simple, the filesystem carries the detail, and no named human appears in an access control list (ACL).
2. Enable access-based enumeration so users see only what their roles permit — the quiet UX of least privilege.
3. Confirm the SMB posture. Current Windows client releases (Pro, Enterprise, and Education) require SMB signing for both inbound and outbound connections, but current Windows Server releases require it only for their own outbound connections — so require signing on SBD-FS01 as well (RequireSecuritySignature in its SMB server configuration), and opt the sensitive share into SMB encryption. Verify both on a live connection from a fleet machine rather than assuming the defaults.
4. Test as members of two different roles from fleet machines: each sees and opens exactly its shares, the other's are invisible and denied, and the denial lands in the pipeline like every other refusal on this network.

Check your result

```
Get-SmbShare | Select-Object Name, Path, FolderEnumerationMode, EncryptData   # on FS01
Get-SmbServerConfiguration | Select-Object RequireSecuritySignature   # on FS01: expected
True
Get-SmbConnection -ServerName SBD-FS01 | Select-Object ShareName, Dialect, Signed,
Encrypted   # on the fleet machine, during the test session
icacls D:\Shares\Projects | Select-String 'SBD'
```

> **WHAT SUCCESS LOOKS LIKE** Shares grant to role groups only, FS01 requires signing, a live connection from the fleet shows Signed True — or, on the encrypted share, Encrypted True, where Signed reads False even though an encrypted connection is also signed — and the two-role test proves both the access and the invisible-denied behavior.

Your record

Evidence to keep: Keep the share and NTFS permission reports, FS01's signing setting and the live connection's signing/encryption output, and the two-role test record with its pipeline entry.

© 2026 Secure By Design Team LLC.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Where does the fine-grained permission detail live?

> **REMEMBER** Roles own the ACLs; humans own role memberships. Signing and encryption are verified on a live connection. Both the access and the denial are on record.

Microsoft · SMB security enhancements

Microsoft · SMB signing

Microsoft · Get-SmbConnection

Microsoft · MSFT_SmbConnection class (Signed and Encrypted)

Microsoft · What's new in Windows Server 2025

LESSON 03 / ABOUT 30 MIN

Build SBD-SQL01 behind its applications

WHY THIS MATTERS The database runs as a managed identity, encrypts its connections with a real certificate, and answers exactly one caller, SBD-APPS01 — the narrowest audience on the network, by design.

In this lesson you will

- Install the database service under its gMSA.
- Bind the SBD Server Authentication certificate for encrypted connections.
- Prove the one-caller rule: SBD-APPS01 connects; SBD-WEB01 and the fleet are refused.

Follow the build

1. Install the database engine on SBD-SQL01 with the lesson-1 gMSA as its service identity — the install itself demonstrates the managed account working where a password account used to live.
2. Bind the issued certificate to the instance and require encrypted connections; the chain validates against pki.sbd.test like every other certificate on this network.
3. Create the application databases with role-based database access for the APPS01 application identity only — least privilege continues inside the engine, not just at its port.
4. Prove the audience: a connection from APPS01 succeeds encrypted. The same attempt from SBD-WEB01 is refused by SBD-SQL01's host firewall — the two share VLAN 40, so the router never sees it — and one from a fleet machine dies at the router with a deny-log entry. Turn on the host firewall's dropped-packet logging if it is off, so both refusals leave a record.

Check your result

```
Get-Service MSSQLSERVER -ComputerName SBD-SQL01 | Select-Object Status
# From SBD-APPS01, the registered caller: expected TcpTestSucceeded True.
Test-NetConnection SBD-SQL01.sbd.test -Port 1433
# From a fleet machine: expected TcpTestSucceeded False.
Test-NetConnection SBD-SQL01.sbd.test -Port 1433
# From SBD-WEB01 (Linux): expected to be refused or time out.
nc -zv SBD-SQL01.sbd.test 1433
certutil -verify -urlfetch C:\Evidence\services\sql01.cer
```

WHAT SUCCESS LOOKS LIKE The service runs as the gMSA, connections are certificate-encrypted with a validating chain, APPS01's access works by database role, and both direct attempts fail with a record: SBD-WEB01's at SQL01's host firewall, the fleet's at the router.

Your record

Evidence to keep: Keep the service identity proof, the certificate binding and urlfetch output, the database role grants, and APPS01's successful connection paired with both refusals and their log entries.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	

RECORD	YOUR OBSERVATION
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What demonstrates the gMSA actually working?

REMEMBER A managed identity runs the engine; no password exists to leak. Connections encrypt under the lab's own validated chain. One caller in: SBD-APPS01. SBD-WEB01 is refused at SQL01's host firewall and the fleet at the router.

Microsoft · Securing SQL Server

Microsoft · Group managed service accounts

LESSON 04 / ABOUT 40 MIN

Build SBD-APPS01 and SBD-WEB01

> **WHY THIS MATTERS** The application server and the internal web server complete the dependency chain: fleet to web, web to application, application to database — each hop a register row, the web tier hardened against its Defense Information Systems Agency (DISA) guidance, and HTTPS served from the lab's own trust.

In this lesson you will

- Stand up SBD-APPS01's licensing service and application API on their registered paths.
- Build the domain-joined Linux Apache server hardened per its DISA Security Technical Implementation Guide (STIG), serving HTTPS with the internal certificate.
- Prove the full chain: fleet → web → application → database, with the web-to-database shortcut refused.

Follow the build

1. Build SBD-APPS01 in the Servers OU and run both of its services under managed identities: the licensing service, verified from the fleet on its registered vendor port, and the application API on HTTPS 443 with its SBD Server Authentication certificate, answering SBD-WEB01 only. APPS01 is the network's one SQL consumer — and the template for any future service: identity, row, evidence.
2. Build SBD-WEB01 on Linux and join it to the sbd.test domain with SSSD, as SBD-SPLUNK01 and SBD-NESSUS01 are joined. Point its packages at SBD-REPO01 only, its syslog over TLS at the collector, and its time service at both domain controllers, and harden the web tier against DISA's Apache guidance — the server's assessable record follows the Linux pattern: hardened configuration exported and hashed, checklist items answered in STIG Viewer.
3. Install the SBD Server Authentication certificate with its chain and serve the internal application over HTTPS — the 2,048-bit key floor from course 1.4's templates applies here as everywhere.
4. Prove the chain from a fleet machine: the HTTPS page loads with a validating certificate, and its data travels fleet → WEB01 → APPS01 → SQL01, every hop a register row. Then prove the shortcuts stay shut: a direct attempt from SBD-WEB01 to SQL01 on 1433 is refused by SQL01's host firewall, and the fleet still cannot touch 1433 at all — three courses of design visible in one browser request.

Check your result

```
Test-NetConnection SBD-WEB01.sbd.test -Port 443   # from a fleet machine
certutil -verify -urlfetch C:\Evidence\services\web01.cer
Test-NetConnection SBD-SQL01.sbd.test -Port 1433   # from the fleet: expected False
# On SBD-WEB01 (Linux): domain membership, the API answers, the database does not.
realm list
curl -sS -o /dev/null -w '%{http_code}\n' https://SBD-APPS01.sbd.test/   # use your
application's health path
nc -zv SBD-SQL01.sbd.test 1433   # expected: refused or timed out
```

> **WHAT SUCCESS LOOKS LIKE** The licensing path works as registered, SBD-WEB01 is domain-joined with SSSD and patches only from SBD-REPO01, the Apache host serves HTTPS with a validating internal chain from a hardened, hashed configuration, and the end-to-end request proves fleet → web → application → database with both shortcuts denied.

Your record

Evidence to keep: Keep APPS01's identity and path proof, WEB01's domain-join and repository settings, its hardened config export with hashes and checklist, the certificate validation output, the end-to-end chain test, and the refused WEB01-to-SQL01 attempt.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Where do WEB01's packages come from?

> **REMEMBER** APPS01 serves licensing and the application API; WEB01 joins the domain like the other Linux servers. The web tier is hardened to its guidance and serves the lab's own trust. One page load proves fleet → web → application → database; the shortcuts stay shut.

Microsoft · Securing SQL Server

DISA · STIGs and GPO package (incl. Apache guidance)

NIST · SP 800-53 Rev. 5 security controls

LESSON 05 / ABOUT 30 MIN

Reconcile, scan, and hand off

> **WHY THIS MATTERS** The business services close the way every build course has: the register reconciled against live behavior, the new systems baselined, the inventory updated — and for the first time, the network is functionally complete.

In this lesson you will

- Reconcile the full register against observed permits and denies.
- Baseline the four service hosts and rescan what changed.
- Update the living records and write the course's handoff.

Follow the build

1. Run the full-register reconciliation one more time: every business-service row proven from its real source, every adjacent deny probed and logged, including WEB01 to SQL01 — the VLAN 40 section of the matrix joins the evidence beside course 1.5's.
2. Run the cadence: SCAP baselines for FS01, SQL01, and APPS01; the config-export record for WEB01; a credentialed scan of VLAN 40 from SBD-NESSUS01 with its findings triaged into the three classifications.
3. Update the living records: the four hosts, their identities, certificates, and rings into the SBD-MGMT01 inventory; the new rows' owners into the register; any service-hardening deviations into the exception record.
4. Write the handoff: every modeled system except the recovery design is now built, watched, patched, and evidenced — course 1.10 proves the network can come back from the dead, and then path 2 takes the whole record to authorization.

Check your result

```
Get-ChildItem C:\Evidence\business-services -Recurse | Select-Object Name, LastWriteTime
Get-FileHash -Algorithm SHA256 C:\Evidence\business-services\*
```

> **WHAT SUCCESS LOOKS LIKE** The matrix covers every VLAN 40 row with paired permit/deny proof, the four hosts carry baselines or config records with triaged scan findings, and inventory, register, and exception record all reflect the finished services.

Your record

Evidence to keep: Finish the checklist: the reconciliation matrix, the scan and triage set, the updated living records, and the handoff note to course 1.10.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	

RECORD	YOUR OBSERVATION
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why re-run the whole register reconciliation now?

REMEMBER Register and reality reconcile across the finished network. Every service host carries its platform-appropriate evidence. One course remains: proving the network can be brought back.

DISA · STIGs and GPO package (incl. Apache guidance)

NIST · SP 800-53 Rev. 5 security controls

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Design the service paths and identities	☐ yes	
02 Build SBD-FS01 with least-privilege shares	☐ yes	
03 Build SBD-SQL01 behind its applications	☐ yes	
04 Build SBD-APPS01 and SBD-WEB01	☐ yes	
05 Reconcile, scan, and hand off	☐ yes	

Open items carried forward

Next in the learning path

That completes course 1.9. Course 1.10 closes the Build path by proving recovery — directory, CA, policy, and logs — before authorization begins.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.